

# Invited: EntroMCU: A Hardware-Algorithm Vision for LLM Inference on Microcontrollers

Arnab Sanyal<sup>†</sup>, Sandeep P. Chinchali<sup>†</sup>, and Gourav Datta<sup>‡</sup>

<sup>†</sup>The University of Texas, Austin, TX  
{sanyal, sandeepc}@utexas.edu

<sup>‡</sup>Case Western Reserve University, Cleveland, OH  
gourav.datta@case.edu

**Abstract**—While small LLMs in the range of 100M–500M parameters have enabled on-device inference using edge GPUs, mobile SoCs, and even Pi-class systems, bare metal microcontroller-based deployment remains open to this day for all models beyond a trivial size. The challenges include up to 1–2 MB of on-chip SRAM, no external DRAM, single in-order cores, low-power budgets of 10–100 mW, and an autoregressive loop causing a large KV cache and full vocabulary LM-head matmul every step. Deployments of LLMs on MCUs have been published so far only for synthetic children’s stories trained with sub-15M-parameter models. In this paper we introduce three specific areas of research leading to deployment of models like Gemma 3 (270M) onto Cortex-M55-class commodity chips. The ideas proposed in this paper cover a ternary GEMM kernel with entropy-encoded weight transport, KV-aware memory management of SRAM/HyperRAM/Flash, and the graph manipulations required in ExecuTorch to put the pieces together.

## I. BACKGROUND

**Challenges of deploying language models on MCU.** The model deployment problem on MCU-class hardware is orders of magnitude behind edge GPU/edge CPU, in terms of scale of deployable models. MCU lacks a DRAM component; its on-chip SRAM capacity is on the order of MB, not GB. An additional computational bottleneck in the deployment problem on MCU comes from the autoregressive decode loop, including a growing KV-cache and a matrix multiplication operation with the full LM tokenizer at every step. There is no hardware-managed cache between different tiers of MCU memory hierarchy; this means the deployment compiler must implement memory hierarchy optimization explicitly. The techniques developed on Jetson class hardware cannot trivially carry over to MCU.

**Current MCU-LLMs.** Only a few academic and enthusiast works explore deploying language models on MCU-class hardware. Deeploy [1], the best MCU deployment framework in academia to date, runs a tiny (2.5M parameters) variant of TinyLlama on Siracusa (an experimental, 16 nm RISC-V MCU with Ethos-U-class NPU). The model is pre-trained on TinyStories [2] synthetic dataset of children’s stories in

constrained vocabulary English. TinyLLM-CIA [3] produces TinyStories class generation results in 100 KB SRAM with an FPU-less MCU (under \$4 price point) using integer attention only. Hobbyist ports of llama2.c [4] can decode 15M parameter TinyStories model at ~19 tokens/s on ESP32-S3 and single-digit tokens/s on Cortex-M7 devices. The state-of-the-art does not reach sub-hundred million parameters on MCU where linguistically useful performance emerges.

## II. OUR VISION FOR MCU-LLMS

**Ternary GEMM kernel for ARM Helium processors.** We propose to implement a ternary GEMM operator for ARM Helium processors. It will employ the highly skewed distribution of post-quantization weights as entropy encoding and ternary computations via lookup tables, combining the two ideas from T-MAC paper [5] and our previous work on entropy-encoding quantization [6], [7]. The new GEMM will perform DMA transfer from Flash, lightweight Huffman decompression and ternary-LUT accumulations concurrently. We will characterize entropy savings for each layer type: attention, feed-forward neural network and embedding layers in sub-billion LLMs. The optimal number of per-tensor versus global codebooks will also be determined under the memory constraint.

**KV cache manager with SRAM/HyperRAM/Flash allocation.** While the KV cache represents the largest memory footprint of the entire decode loop, it is currently treated like short-term activations by the majority of MCUs runtimes [8]. We envision developing a novel KV cache manager that treats the cache as a long-lived object and allocates SRAM/HyperRAM/Flash resources for it accordingly. Our technique will combine sliding window attention, chunked prefill and FlashAttention style computation to make use of limited SRAM and stream larger windows from external memory. A constraint programming solution will generate static memory allocation plans to be incorporated into ExecuTorch.

**Graph-level optimization and ExecuTorch integration.** We propose to create several graph transformations, aiming to reduce the inference cost and model size on MCU-class hardware. The first one is pruning of unused vocabulary from the multilingual language models, to decrease overheads related to embedding and LM-head operations in English-language deployments. After transformation, the model will go through our proposed quantization and compiler pass using our own set of ExecuTorch operators, among which is the ternary GEMM and tiled attention operators described above. We plan to augment ExecuTorch with multi-tier memory planning algorithms and execute heterogeneous (MCU-NPU) models.

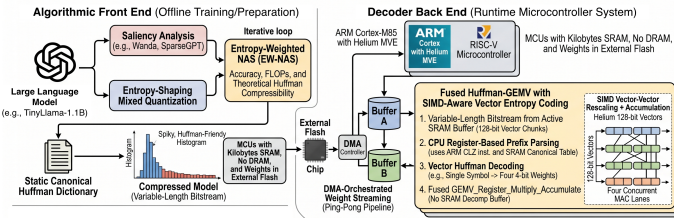


Fig. 1. Proposed hardware–algorithm pipeline for MCU-based LLM inference. Quantized weights are Huffman-encoded and stored in Flash; at runtime, a fused Huffman-GEMV engine streams, decodes, and processes weights through DMA, SRAM buffering, and ARM Helium SIMD execution.

## REFERENCES

- [1] M. Scherer *et al.*, “Deeploy: Enabling energy-efficient deployment of small language models on heterogeneous microcontrollers,” *IEEE TCAD*, vol. 43, no. 11, pp. 4009–4020, 2024.
- [2] R. Eldan and Y. Li, “TinyStories: How small can language models be and still speak coherent English?” *arXiv preprint arXiv:2305.07759*, 2023.
- [3] Y. Suzuki, “TinyLLM-CIA: Language model inference in 100 kb RAM on a \$4 microcontroller without floating-point unit,” Preprint, Feb. 2026.
- [4] A. Karpathy, “llama2.c: Inference of Llama 2 in one file of pure C,” <https://github.com/karpathy/llama2.c>, 2024.
- [5] J. Wei *et al.*, “T-MAC: CPU Renaissance via Table Lookup for Low-Bit LLM Deployment on Edge,” in *Proceedings of the Twentieth European Conference on Computer Systems*, 2025, p. 278–292. [Online]. Available: <https://doi.org/10.1145/3689031.3696099>
- [6] A. Sanyal, G. Datta, P. Mukherjee, S. P. Chinchali, and M. Orshansky, “Entrollm: Entropy encoded weight compression for efficient large language model inference on edge devices,” in *ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026, pp. 19 742–19 746.
- [7] C. Zhou, S. Sarkar, Y. Li, A. Sanyal, and G. Datta, “OASIS: Optimized Lightweight Autoencoder System for Distributed In-Sensor computing,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.02256>
- [8] M. AI. (2025) ExecuTorch: On-device AI inference framework for PyTorch models. Accessed: Apr. 30, 2026. [Online]. Available: <https://pytorch.org/executorch>